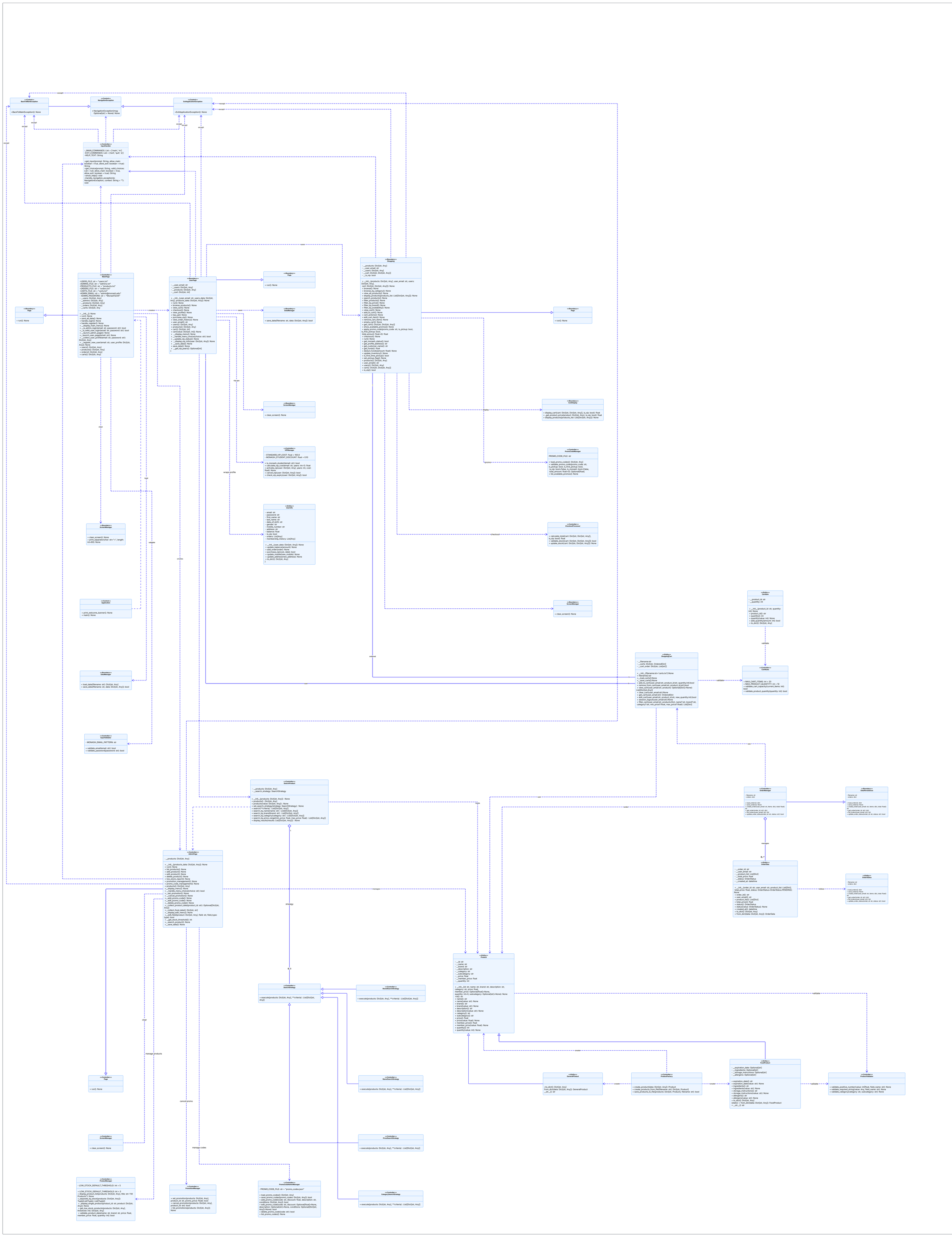
<<Controller>>
BackToMainException
+BackToMainException(): None
<<Controller>>
NavigationException
+ NavigationException(msg: Optional[str] = None): None
<<Controller>>
ExitApplicationException
+ExitApplicationException(): None
<<Controller>>
InputHandler
- _MAIN_COMMANDS: List = ['main', 'm']
-EXIT_COMMANDS: List = ['exit', 'quit', 'q']
-HELP_TEXT: String
+get_input(prompt: String, allow_main: boolean = true, allow_exit: boolean = true): String
+get_choice(prompt: String, valid_choices: List = null, allow_main: boolean = true, allow_exit: boolean = true): String
+show_help(): void
+handle_navigation_exception(e: NavigationException, context: String = ""): void
<<Controller>>
MainPage
-USERS_FILE: str = "users.txt"
-ADMINS_FILE: str = "admins.txt"
-PRODUCTS_FILE: str = "products.txt"
-ORDERS_FILE: str = "orders.txt"
-CARTS_FILE: str = "carts.txt"
-ADMIN_EMAIL: str = "admin@monash.edu"
- ADMIN_PASSWORD: str = "Monash1234!"
- __user_page: UserPage
- __admins: Dict[str, Any]
- __products: Dict[str, Any]
- __orders: Dict[str, Any]
- __carts: Dict[str, Any
+ __init__(): None
+ run(): None
+ save_all_data(): None
+ handle_login(): None
+ handle_register(): None
+ __display_main_menu(): None
+ __is_admin_login(email: str, password: str): bool
+ __handle_admin_login(email: str, password: str): bool
+ __launch_admin_page(): None
+__launch_user_page(email: str): None
+ __collect_user_profile(email: str, password: str): Dict[str, Any]
+ __register_new_user(email: str, user_profile: Dict[str, Any]): None
+ users(): Dict[str, Any]
+ products(): Dict[str, Any]
+ orders(): Dict[str, Any]
+ carts(): Dict[str, Any]
<<Boundary>>
Page
+ run(): None
<<Boundary>>
UserPage
- __user_email: str
- __users: Dict[str, Any]
- __products: Dict[str, Any]
- __cart: Dict[str, int]
+ __init__(user_email: str, users_data: Dict[str, Any], products_data: Dict[str, Any]): None
+ run(): None
+ browse_products(): None
+ view_cart(): None
+ checkout(): None
+ view_profile(): None
+ top_up(): None
+ purchase_vip(): None
+ view_vip_status(): None
+ user_email(): str
+ users(): Dict[str, Any]
+ products(): Dict[str, Any]
+ cart(): Dict[str, int]
+ cart(value: Dict[str, int]): None
+ __display_menu(): None
+ __handle_menu_choice(choice: str): bool
+ __update_vip_status(): None
+ __display_vip_info(user: Dict[str, Any]): None
+ __save_data(): None
+ __view_data(): None
+ __get_vip_years(): Optional[int]
}
<<Boundary>>
DataManager
+ load_data(filename: str): Dict[str, Any]
+ save_data(filename: str, data: Dict[str, Any]): bool
<<Controller>>
InputValidator
- MONASH_EMAIL_PATTERN: str
+ validate_email(email: str): bool
+ validate_password(password: str): bool
<<Boundary>>
ScreenManager
+ clear_screen(): None
+ print_separator(char: str="=", length: int=60): None
<<Controller>>
Application
+ print_welcome_banner(): None
+ main(): None
<<Controller>>
Page
+ run(): None
<<Controller>>
ScreenManager
+ clear_screen(): None
<<Controller>>
ProductManager
- LOW_STOCK_DEFAULT_THRESHOLD: int = 5
+ LOW_STOCK_DEFAULT_THRESHOLD: int = 5
+ display_product_list(products: Dict[str, Any], title: str="All Products"): None
+ separate_by_stock(products: Dict[str, Any]): Tuple[List[Tuple], List[Tuple]]
+ _display_single_product(product_id: str, product: Dict[str, Any]): None
+ get_low_stock_products(products: Dict[str, Any], threshold: int): Dict[str, Any]
+ validate_product_data(name: str, brand: str, price: float, member_price: float, quantity: int): bool
<<Boundary>>
Page
+ run(): None
<<Boundary>>
EvaMenager
+ save_data(filename: str, data: Dict[str, Any]): bool
<<Boundary>>
ScreenManager
+ clear_screen(): None
<<Controller>>
VIPManager
- STANDARD_VIP_COST: float = 100.0
- MONASH_STUDENT_DISCOUNT: float = 0.30
+ is_monash_student(email: str): bool
+ calculate_vip_cost(email: str, years: int=1): float
+ activate_vip(user: Dict[str, Any], years: int, cost: float): None
+ cancel_vip(user: Dict[str, Any]): bool
+ check_vip_expiry(user: Dict[str, Any]): bool
<<Entity>>
UserInfo
- email: str
- password: str
- first_name: str
- last_name: str
- date_of_birth: str
- gender: str
- mobile_number: str
- address: str
- balance: float
- is_vip: bool
- orders: List[Any]
- membership_history: List[Any]
+ __init__(user_data: Dict[str, Any]): None
+ update_balance(amount): None
+ top_up(amount): None
+ purchase_vip(cost, date): bool
+ update_mobile(new_mobile): None
+ update_address(new_address): None
+ to_dict(): Dict[str, Any]
}
<<Boundary>>
Shopping
- __products: Dict[str, Any]
- __user_email: str
- __users: Dict[str, Any]
- __cart: Dict[str, Dict[str, Any]]
- __is_vip: bool
+ __init__(products: Dict[str, Any], user_email: str, users: Dict[str, Any], cart: Dict[str, Dict[str, Any]]): None
+ browse(): None
+ browse_by_category(): None
+ view_all_products(): None
+ display_products(products_list: List[Dict[str, Any]]): None
+ sort_products(): None
+ filter_products(): None
+ filter_by_price(): None
+ filter_by_brand(): None
+ filter_by_availability(): None
+ view_cart(): None
+ add_to_cart(): None
+ cart_actions(): None
+ edit_cart_item(): None
+ remove_cart_item(): None
+ empty_cart(): None
+ get_cart(): Dict[str, Dict[str, Any]]
+ show_available_promos(): None
+ apply_promo_code(promo_code: str, is_pickup: bool, is_first_pickup: bool, total_amount: float=0): float
+ checkout(): None
+ run(): None
+ get_vip_status(): bool
+ get_profile_address(): str
+ get_profile(prompt_name): str
+ get_funds(): float
+ deduct_funds(amount: float): None
+ update_inventory(): None
+ is_first_time_pickup(): bool
+ set_pickup_flag(): None
+ products(): Dict[str, Any]
+ user_email(): str
+ users(): Dict[str, Any]
+ cart(): Dict[str, Dict[str, Any]]
+ is_vip(): bool
<<Boundary>>
Page
+ run(): None
<<Boundary>>
CartDisplay
+ display_cart(cart: Dict[str, Dict[str, Any]], is_vip: bool): float
+ _get_product_price(product: Dict[str, Any], is_vip: bool): float
+ display_products(products_list: List[Dict[str, Any]]): None
<<Controller>>
PromoCodeManager
PROMO_CODE_FILE: str
+ load_promo_codes(): Dict[str, Any]
+ validate_promo_code(promo_code: str, is_pickup: bool, is_first_pickup: bool=False, is_vip: bool=False, total_amount: float=0): Optional[float]
+ list_available_promos(): None
<<Controller>>
CheckoutProcessor
+ calculate_total(cart: Dict[str, Dict[str, Any]], is_vip: bool): float
+ validate_stock(cart: Dict[str, Dict[str, Any]]): bool
+ update_stock(cart: Dict[str, Dict[str, Any]]): None
<<Boundary>>
ScreenManager
+ clear_screen(): None
<<Entity>>
CartItem
- __product_id: str
- __quantity: int
+ __init__(product_id: str, quantity: int): None
+ product_id(): str
+ quantity(): int
+ quantity(value: int): None
+ add_quantity(amount: int): bool
+ to_dict(): Dict[str, Any]
<<Entity>>
ShoppingCart
- __filename:str
- __carts: Dict[str, OrderedDict]
- __cart_order: Dict[str, List[str]]
+ __init__(filename:str='carts.txt'):None
+ filename():str
+ __load_carts():None
+ __save_carts():None
+ add_to_cart(user_email:str, product_id:str, quantity:int):bool
+ remove_from_cart(user_email:str, product_id:str):bool
+ view_cart(user_email:str, products: Optional[Dict]=None): List[Dict[str,Any]]
+ clear_cart(user_email:str):None
+ get_cart(user_email:str): OrderedDict
+ edit_cart(user_email:str, product_id:str, new_quantity:int):bool
+ empty_cart(user_email:str):None
+ filter_cart(user_email:str, products:Dict, name?:str, brand?:str, category?:str, min_price?:float, max_price?:float): List[Dict]
<<Controller>>
CartRules
+ MAX_CART_ITEMS: int = 20
+ MAX_PRODUCT_QUANTITY: int = 10
+ validate_cart_capacity(current_items: int): bool
+ validate_product_quantity(quantity: int): bool
<<Controller>>
SearchProduct
- __products: Dict[str, Any]
- __search_strategy: SearchStrategy
+ __init__(products: Dict[str, Any]) : None
+ products() : Dict[str, Any]
+ products(value: Dict[str, Any]) : None
+ set_search_strategy(strategy: SearchStrategy) : None
+ search(**criteria) : List[Dict[str, Any]]
+ search_by_name(name: str) : List[Dict[str, Any]]
+ search_by_brand(brand: str) : List[Dict[str, Any]]
+ search_by_category(category: str) : List[Dict[str, Any]]
+ search_by_price_range(min_price: float, max_price: float) : List[Dict[str, Any]]
+ display_results(results: List[Dict[str, Any]]) : None
<<Controller>>
OrderManager
- filename: str
- orders: dict
+ load_orders(): dict
+ save_orders(): None
+ create_order(user_email: str, items: dict, total: float): str
+ get_order(order_id: str): dict
+ list_orders(user_email: str): list
+ update_order_status(order_id: str, status: str): bool
<<Boundary>>
DataPersistence
- filename: str
- orders: dict
+ load_orders(): dict
+ save_orders(): None
+ create_order(user_email: str, items: dict, total: float): str
+ get_order(order_id: str): dict
+ list_orders(user_email: str): list
+ update_order_status(order_id: str, status: str): bool
<<Entity>>
OrderData
- __order_id: str
- __user_email: str
- __product_list: List[Dict]
- __total_price: float
- __status: OrderStatus
- __created_at: datetime
+ __init__(order_id: str, user_email: str, product_list: List[Dict], total_price: float, status: OrderStatus=OrderStatus.PENDING): None
+ order_id(): str
+ user_email(): str
+ product_list(): List[Dict]
+ total_price(): float
+ status(): OrderStatus
+ status(value: OrderStatus): None
+ created_at(): datetime
+ to_dict(): Dict[str, Any]
+ from_dict(data: Dict[str, Any]): OrderData
<<Entity>>
OrderStatus
- filename: str
- orders: dict
+ load_orders(): dict
+ save_orders(): None
+ create_order(user_email: str, items: dict, total: float): str
+ get_order(order_id: str): dict
+ list_orders(user_email: str): list
+ update_order_status(order_id: str, status: str): bool
<<Controller>>
AdminPage
- __products: Dict[str, Any]
+ __init__(products_data: Dict[str, Any]): None
+ run(): None
+ add_product(): None
+ edit_product(): None
+ delete_product(): None
+ low_stock_report(): None
+ promotion_management(): None
+ promo_code_management(): None
+ products(): Dict[str, Any]
+ __display_menu(): None
+ __handle_menu_choice(choice: str): bool
+ __set_promotion(): None
+ __cancel_promotion(): None
+ __add_promo_code(): None
+ __edit_promo_code(): None
+ __delete_promo_code(): None
+ __collect_product_data(product_id: str): Optional[Dict[str, Any]]
+ __collect_food_data(): Dict[str, str]
+ __display_edit_menu(): None
+ __edit_field(product: Dict[str, Any], field: str, field_type: str): bool
+ __get_stock_threshold(): int
+ __search_product(): None
+ __save_data(): None
<<Controller>>
SearchStrategy
+ execute(products: Dict[str, Any], **criteria) : List[Dict[str, Any]]
<<Controller>>
BrandSearchStrategy
+ execute(products: Dict[str, Any], **criteria) : List[Dict[str, Any]]
<<Controller>>
NameSearchStrategy
+ execute(products: Dict[str, Any], **criteria) : List[Dict[str, Any]]
<<Controller>>
ProdcutSearchStrategy
+ execute(products: Dict[str, Any], **criteria) : List[Dict[str, Any]]
<<Controller>>
CategorySearchStrategy
+ execute(products: Dict[str, Any], **criteria) : List[Dict[str, Any]]
<<Controller>>
PromotionManager
+ set_promotion(products: Dict[str, Any], product_id: str, promo_price: float): bool
+ cancel_promotion(products: Dict[str, Any], product_id: str): bool
+ list_promotions(products: Dict[str, Any]): None
<<Controller>>
PromoCodeAdminManager
- PROMO_CODE_FILE: str = "promo_codes.json"
+ load_promo_codes(): Dict[str, Any]
+ save_promo_codes(promo_codes: Dict[str, Any]): bool
+ add_promo_code(code: str, discount: float, description: str, conditions: Optional[Dict[str, Any]]): bool
+ edit_promo_code(code: str, discount: Optional[float]=None, description: Optional[str]=None, conditions: Optional[Dict[str, Any]]=None): bool
+ delete_promo_code(code: str): bool
+ list_promo_codes(): None
<<Entity>>
Product
- __id: str
- __name: str
- __brand: str
- __description: str
- __category: str
- __subcategory: str
- __price: float
- __member_price: float
- __quantity: int
+ __init__(id: str, name: str, brand: str, description: str, category: str, price: float, member_price: Optional[float]=None, quantity: int=0, subcategory: Optional[str]=None): None
+id(): str
+ name(): str
+ name(value: str): None
+ brand(): str
+ brand(value: str): None
+ description(): str
+ description(value: str): None
+ category(): str
+ subcategory(): str
+ price(): float
+ price(value: float): None
+ member_price(): float
+ member_price(value: float): None
+ quantity(): int
+ quantity(value: int): None
<<Entity>>
GeneralProduct
+to_dict(): Dict[str, Any]
from_dict(data: Dict[str, Any]): GeneralProduct
+__str__(): str
<<Controller>>
ProductFactory
+ create_product(data: Dict[str, Any]): Product
+ create_products_from_file(filename: str): Dict[str, Product]
+ save_products_to_file(products: Dict[str, Product], filename: str): bool
<<Entity>>
FoodProduct
- __expiration_date: Optional[str]
- __ingredients: Optional[str]
- __storage_instructions: Optional[str]
- __allergens: Optional[str]
+ expiration_date(): str
+ expiration_date(value: str): None
+ ingredients(): str
+ ingredients(value: str): None
+ storage_instructions(): str
+ storage_instructions(value: str): None
+ allergens(): str
+ allergens(value: str): None
+ to_dict(): Dict[str, Any]
[static] + from_dict(data: Dict[str, Any]): FoodProduct
+ __str__(): str
<<Controller>>
ProductValidator
+ validate_positive_number(value: int|float, field_name: str): None
+ validate_required_string(value: Any, field_name: str): None
+ validate_category(category: str, subcategory: str): None
except
except
except